CMDBUF Studio
FSDB 波形转 Formal 操作说明

2026 11 11 第 34 份说明 适用于当前 CMDBUF 接口

新增“波形转 Formal”页面，可导入本机 FSDB，也可选择回归生成的 FSDB。工具在 VM 中调用 Verdi NPI，直接提取复位后的每拍输入输出，生成以 pattern 区分用例的 assume、assert 与终点 cover。提取过程不依赖 VCD、TC 源码或 RTL 数据库。

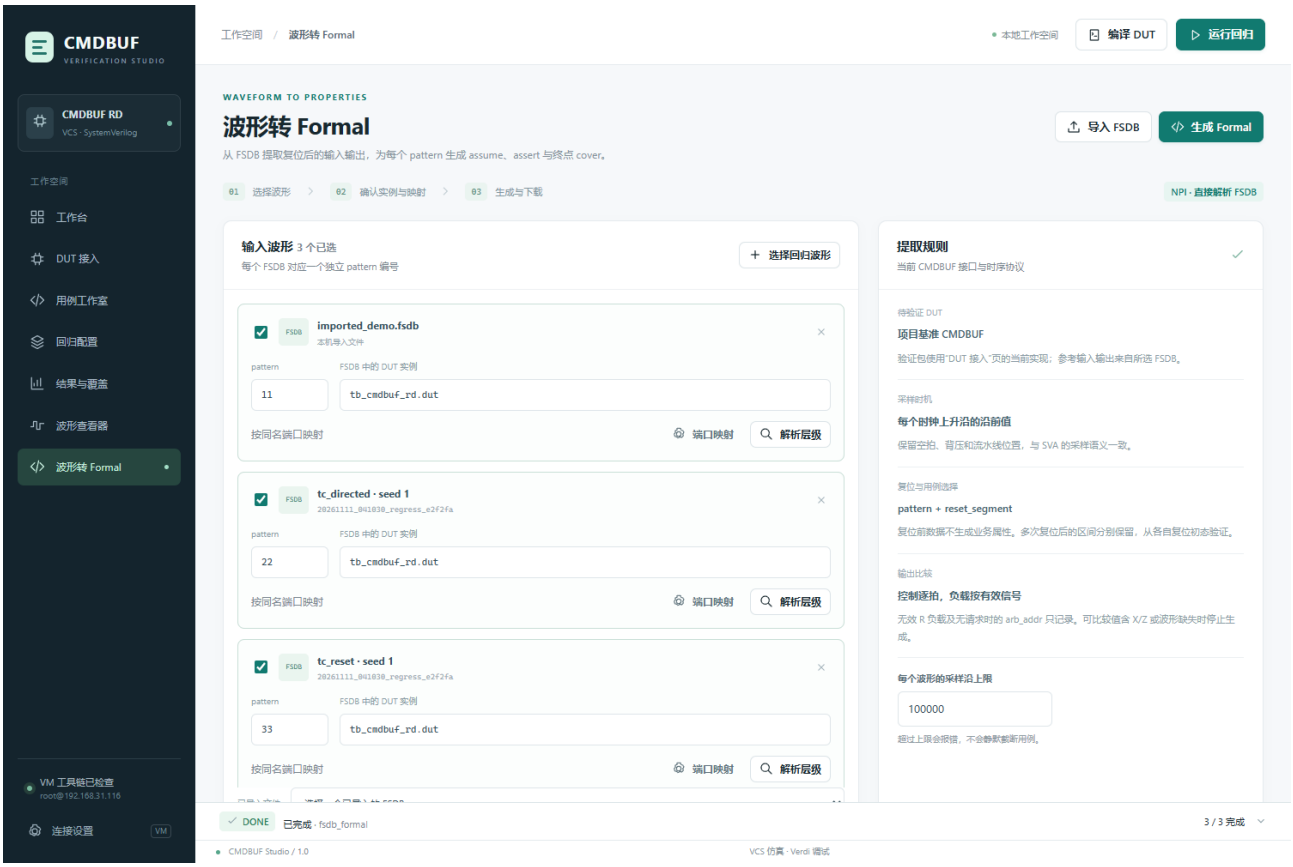


图 1 实际界面 一个导入波形与两个回归波形分别使用 pattern 11 22 33

1 启动与生成

```
.\make.cmd gui
```

也可双击项目根目录的 CMDBUF Studio.cmd。进入“波形转 Formal”，点击“导入 FSDB”或“选择回归波形”，勾选来源，填写互不重复的 pattern。确认 DUT 层级及端口映射后，点击“生成 Formal”，完成后下载完整验证包。

参考数据来自所选 FSDB；验证包中的待验证实现取自“DUT 接入”页的当前 RTL 和例化。要验证另一个 CMDBUF 实现，先在该页保存接入配置，再从参考波形生成验证包。

2 确认层级和采样语义

默认实例为 tb_cmdbuf_rd.dut。若外部 FSDB 的 testbench 或 DUT 名称不同，点击“解析层级”，从发现的实际层级中选择目标实例；默认层级不存在也会显示可选层级。端口同名时自动映射，名称不同可在“端口映射”中填写完整的 FSDB 信号路径。

当前映射检查 31 个标量或数组元素，包含时钟、复位和 8 个 Bank 的 rd_mem_status[0] 至 [7]。位宽必须一致。rd_mem_status 保留四态二维接口，128 位 MEM/R 数据不会截短。回归中未开启波形的运行不能作为 FSDB 来源。

每一个周期都保留

发现 clk 从 0 变为 1 的时间 T 后，NPI 查询 T 之前一个 FSDB 时间刻度的值。这与 SVA 在上升沿采样沿前值的语义对齐，避免把 DUT 在同一时刻更新后的输出误当作本拍输出。时钟作为采样事件使用，不生成逐位时钟 assume；时钟暂停期间没有采样拍。

复位拉低阶段不生成业务属性。rst_n 释放后的第一个上升沿记为 trace_cycle=0；后续每个上升沿递增。空拍、背压和流水线等待均保留，不把相隔多拍的握手压缩到相邻周期。AR、ISSUE、R 握手另存为事务清单，包含原周期号和 FSDB 时间。

一个波形一个 pattern

pattern 是 Formal 顶层的符号输入。生成器限制它只能选择已导出的编号，并在一个复位后区间内保持不变。每份 FSDB 的多个复位后区间使用 reset_segment 区分，从各自的复位初态独立验证，区间内的输入输出顺序完整保留。

pattern	来源	reset_segment 与周期数
11	导入 imported_demo.fsdb	0：69 拍
22	tc_directed seed 1	0：69 拍
33	tc_reset seed 1	0：5 拍；1：9 拍；2：53 拍

例如 pattern=33、reset_segment=1 选择第二个复位后的 9 拍；不是把三次复位前后的数据拼成一个连续的 67 拍序列。多段方案检查各段复位后的业务行为，不专门验证复位过程或跨复位状态保留。

3 生成的 assume 和 assert

下面来自本轮实际导出包的第 0 拍；仅合并换行。每个周期分别生成一条包含全部输入的 assume 与一条包含应比较输出的 assert，标签保留 pattern、复位区间和周期号，便于从反例回查原 FSDB。

```
M_P11_S0_C00000: assume property (
  @(posedge clk) disable iff (!rst_n)
  (pattern == 32'd11 && reset_segment == 32'd0 &&
   trace_cycle == 32'd0) |-> (
    rst_n == 1'h1 && arvalid == 1'h1 &&
    araddr == 18'h12001 && src == 2'h2 &&
    arid == 8'h37 && ar_port_id == 5'h06 &&
    arresp == 2'h0 && rready == 3'h7 &&
    ar_rd_gnt == 8'hFF && rw_rd_gnt == 8'hFF &&
    rd_mem_status[0] == 2'h0 && rd_mem_status[1] == 2'h0 &&
    rd_mem_status[2] == 2'h0 && rd_mem_status[3] == 2'h0 &&
    rd_mem_status[4] == 2'h0 && rd_mem_status[5] == 2'h0 &&
    rd_mem_status[6] == 2'h0 && rd_mem_status[7] == 2'h0 &&
    mem_rdata == 128'h00000000000000000000000000000000 &&
    pc_err == 1'h0));

A_P11_S0_C00000: assert property (
  @(posedge clk) disable iff (!rst_n)
  (pattern == 32'd11 && reset_segment == 32'd0 &&
   trace_cycle == 32'd0) |-> (
    arready == 3'h7 && rvalid == 3'h0 &&
    req == 8'h00 && pending_cnt == 4'h1));
```

第一条约束 Formal 在该用例第 0 拍使用波形中的完整输入。第二条要求同拍输出与参考值一致。此拍 rvalid 和 req 都是 0，所以断言没有比较无效 R 负载和 arb_addr；有效周期会自动增加这些字段。

检查用例确实走到末尾

```
C_END_P11_S0: cover property (
  @(posedge clk) disable iff (!rst_n)
  pattern == 32'd11 && reset_segment == 32'd0 &&
  trace_cycle == 32'd68);
```

每个区间都有独立终点 cover。assert 通过后还要检查全部终点命中，以排除输入假设矛盾、深度不足或一直处于复位导致的空通过。若只想检查指定用例，可在目标 Formal 工具额外固定 pattern 和 reset_segment；相应只要求所选终点 cover。

当前代码不使用 started、active 或复位序列 assume。所有原生属性统一采用 @(posedge clk) disable iff (!rst_n)，建立 DUT 复位初态由 Formal 工具配置完成。

4 下载后运行 Formal

完整 ZIP 包包含检查器、当前 DUT 快照、工具配置和原始提取数据。界面“已生成”只表示生成完成；本次交付额外运行的 Formal 结果保存在独立验收记录中，不把生成成功显示成证明通过。

文件	用途
trace_properties.sv	便于阅读的原生 assume / assert / cover。
formal_top.sv 与 files.f	当前 DUT 例化、检查器连接以及原生源码列表。
rtl_snapshot/	生成时选中的 DUT RTL 快照，可追溯实际验证实现。
trace.json 与 samples.csv	全部采样数据，含原周期、时间和四态值。
transactions.json	AR / ISSUE / R 握手，保留周期和时间位置。
integration.json	时钟、复位初始化、有效负载规则与运行方法。
Makefile 与 run_*.sby	已实测的 SBY 有界检查和终点覆盖运行入口。
sby_properties.sv 与 sby_adaptation.json	SBY 执行副本和转换说明；原生文件保持不变。
SHA256SUMS.json	文件哈希，用于核对生成包内容。

通过 Make 提取和运行

GUI 的解析与生成按钮都会在 VM 调用下面的 Make 目标。REQUEST 指向本次任务的 JSON 清单，完整路径可在任务日志查看；GUI 已负责创建、同步及上传，不需要用户手写清单。

```
make fsdb-inspect REQUEST=/path/to/request.json
make fsdb-formal REQUEST=/path/to/request.json
```

解压验证包，进入包内目录，在已配置好 Yosys、read_slang、SBY、ABC 和 Yices 的环境执行：

```
make bmc
make cover
```

包内深度按最长区间加 2 自动生成。本例最长 69 拍，深度 71。迁移到其他 Formal 工具时加载原生 files.f，设置 cmdbuf_fsdb_formal_top 为顶层、clk 为时钟，并执行真实 rst_n 复位初始化，再运行 assert 和终点 cover。

5 实测结果

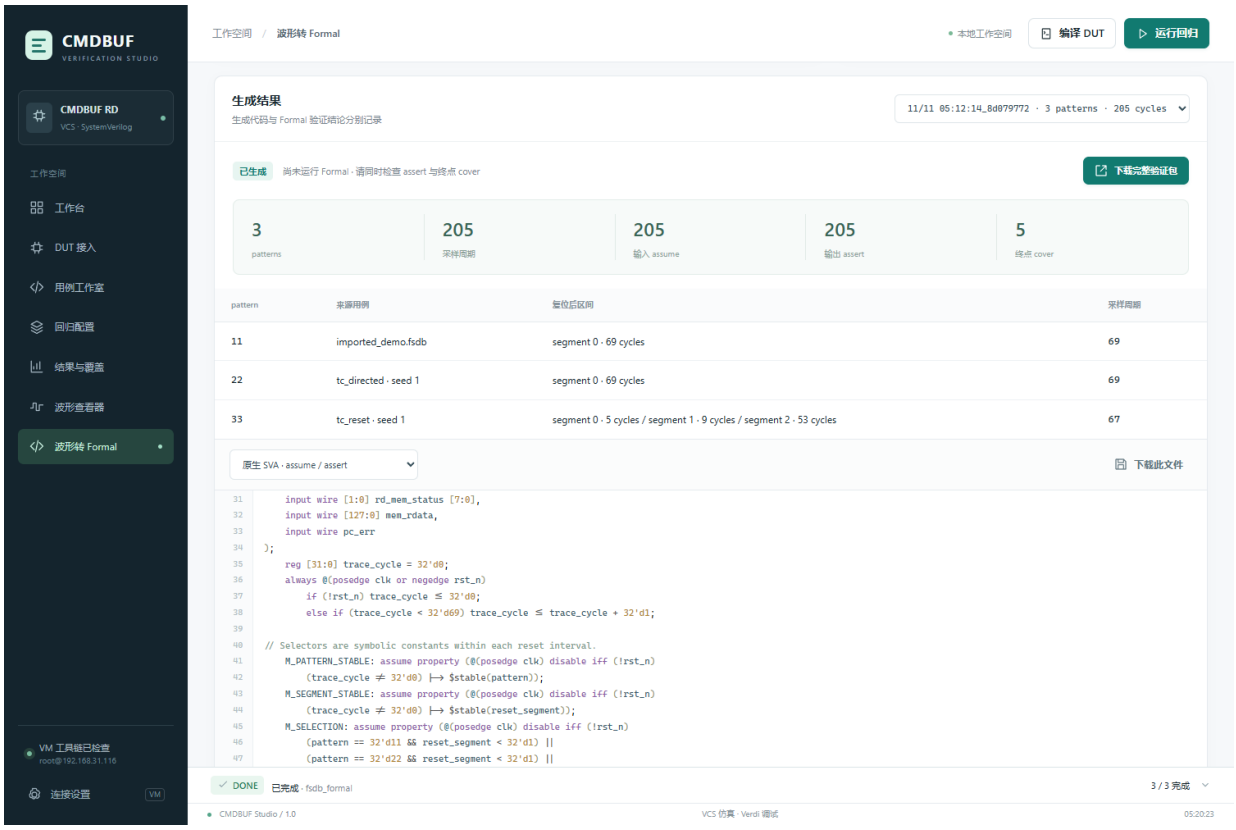


图 2 实际生成结果 3 个 pattern 205 个采样周期 5 个终点 cover

验证项目	本轮结果
直接 NPI 提取	3 个 FSDB，5 个复位后区间，205 拍，13 个握手记录。
独立数值核对	6,355 个端口值全部一致；60 个非时钟值在沿后变化。
VCS 与 Formal	原生 SVA 重放通过；71 步 BMC 通过；5 个终点全部命中。
注错与工具检查	输出注错和 Formal 断言篡改均检出；VM 单元测试 35/35。

实际操作覆盖了本机文件导入、回归波形选择、未知默认层级的发现与切换、31 项端口映射、不同 pattern、代码和 CSV 预览、完整 ZIP 下载。1500×1000、1280×800、1100×720 三种窗口尺寸均无页面横向溢出。

验收记录位于 verification/fsdb_formal_20261111/acceptance.json，原始运行日志、终点 cover 轨迹、注错反例和实际下载的验证包一起留存。全部 8 轮历史签名保持不变；本次没有修改 RTL 或规范 SVA。

6 数据要求与工具迁移

VM 需要可用的 Verdi NPI 头文件、libNPI.so、C++ 编译器和现有 Make/SSH 环境。工具按 VM 安装路径编译读取器，不在导出包中分发厂商库。导入的波形须记录完整的 CMDBUF 端口、时钟和低有效复位。

检查项	处理方法
缺少信号或位宽不符	确认 DUT 层级和映射；仍缺少时重新导出包含全部接口的 FSDB。
没有复位或历史不完整	重新记录从复位开始的波形；业务区间内存在 dump-off 会停止生成。
时钟或复位边沿不明确	拒绝含 X/Z 的时钟边沿、同时间多次上升沿，以及复位在采样沿切换。
超出采样上限	在页面明确调高上限后重新生成；默认 100000，最大 1000000。
重复 pattern 或无波形运行	修改为唯一编号；回归必须开启波形并真实生成 FSDB。

输入和输出的处理

每拍的全部非时钟输入写入 assume，控制输出逐拍写入 assert。依据项目已有协议，rvalid=0 时的 R 负载，以及 req=0 时的 arb_addr 没有定义值，仍保存在原始数据中，但不写入比较断言。有效输出和输入包含 X/Z 时停止生成；不会擅自替换为 0。

建立真实的复位初态

disable iff 只禁用复位期间的属性，不会把 DUT 状态初始化。附带 SBY 配置用实际复位仿真建立初态；没有把所有状态强行清零。SBY 前端的同拍蕴含和 $stable 适配仅写入执行副本，并记录源文件、转换规则及哈希。其他 Formal 工具须配置相应的复位初始化流程。

验证结论的范围

本功能验证已记录的有限输入轨迹下，当前 DUT 输出是否符合参考波形。参考波形本身不是规格证明；此结果不表示覆盖任意输入或无界证明整个协议。另一个实现若有不同的合法流水线延迟，也会与此逐拍参考产生差异。

接口实现依据提供的 VC_APPS_NPI.pdf 中 NPI 初始化、构建及 FSDB 章节，重点见 PDF 第 12 至 16、142、155 至 158 页。提取器使用 FSDB 数据接口；验收中的 VCD 仅作独立数值核对，不是此功能的运行依赖。